pràctica_3

March 3, 2026

1 Lectura de datos y tratamiento

[1]:
```
import keras
from keras import layers
```

```
2026-03-03 09:45:23.985169: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1772527523.998602   54362 cuda_dnn.cc:8310] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1772527524.005691   54362 cuda_blas.cc:1418] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
2026-03-03 09:45:24.029186: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
```

[2]:
```
import pandas as pd
clinical = pd.read_csv('clinical.csv',sep="\t",
    escapechar="\\",
    skipinitialspace=True)
clinical.head()
```

[2]:
```
            Sample                      Histology PAM50Call  \
0  TCGA.A1.A0SB.01                            NaN       NaN
1  TCGA.A1.A0SD.01  Infiltrating Ductal Carcinoma      LumA
2  TCGA.A1.A0SE.01                 Mixed Histology      LumA
3  TCGA.A1.A0SF.01                            NaN       NaN
4  TCGA.A1.A0SG.01                            NaN       NaN

  ajcc_cancer_metastasis_stage_code ajcc_neoplasm_disease_lymph_node_stage  \
0                                M0                                     N0
```

```
1                                M0                                      N0
2                                M0                               N0 (i-)
3                                M0                                      N0
4                                M0                                     N1a

  ajcc_neoplasm_disease_stage ajcc_tumor_stage_code  \
0                     Stage I                   T1c
1                   Stage IIA                    T2
2                     Stage I                   T1c
3                   Stage IIA                    T2
4                   Stage IIB                    T2

  breast_cancer_optical_measurement_histologic_type  \
0                                            Other
1                              Infiltrating Ductal
2                                  Mixed Histology
3                              Infiltrating Ductal
4                                            Other

  breast_carcinoma_estrogen_receptor_status  \
0                                 Positive
1                                 Positive
2                                 Positive
3                                 Positive
4                                 Positive

  breast_carcinoma_progesterone_receptor_status  \
0                                     Negative
1                                     Positive
2                                     Positive
3                                     Positive
4                                     Positive

  breast_carcinoma_surgical_procedure_name  gender  \
0                              Lumpectomy  FEMALE
1                              Lumpectomy  FEMALE
2              Modified Radical Mastectomy  FEMALE
3              Modified Radical Mastectomy  FEMALE
4                              Lumpectomy  FEMALE

  her2_immunohistochemistry_level_result  \
0                                    NaN
1                                    NaN
2                                     1+
3                                    NaN
4                                    NaN
```

```
  lab_proc_her2_neu_immunohistochemistry_receptor_status margin_status  \
0                                           Negative          Negative
1                                           Negative          Positive
2                                           Negative          Negative
3                                           Negative          Negative
4                                           Negative          Negative

                                   menopause_status  \
0  Post (prior bilateral ovariectomy OR >12 mo si…
1                                               NaN
2  Pre (<6 months since LMP AND no prior bilatera…
3  Pre (<6 months since LMP AND no prior bilatera…
4  Post (prior bilateral ovariectomy OR >12 mo si…

  person_neoplasm_cancer_status    sample_type vital_status
0                    TUMOR FREE  Primary Tumor       LIVING
1                           NaN  Primary Tumor       LIVING
2                    TUMOR FREE  Primary Tumor       LIVING
3                    TUMOR FREE  Primary Tumor       LIVING
4                           NaN  Primary Tumor       LIVING
```

[3]:
```python
protein_abundance = pd.read_csv('protein_abundance.csv',sep="\t",
    escapechar="\\",
    skipinitialspace=True)
protein_abundance.head()
```

[3]:
```
             Sample  14-3-3_epsilon    4E-BP1  4E-BP1_pS65  4E-BP1_pT37  \
0  TCGA.A1.A0SH.01       -0.214638 -0.034871    -0.149239    -0.114105
1  TCGA.A1.A0SJ.01        0.134356  0.239884    -0.173064    -0.878286
2  TCGA.A1.A0SK.01        0.218652  2.348938     0.479825     1.280368
3  TCGA.A1.A0SO.01       -0.112494  0.278607     0.177655     0.672807
4  TCGA.A2.A04N.01        0.002115  0.708655    -0.206350     0.649848

   4E-BP1_pT70     53BP1  A-Raf_pS299      ACC1  ACC_pS79  …  mTOR_pS2448  \
0     0.188328 -0.230038     0.216368  1.361227  1.512532  …     0.201216
1    -0.166205 -0.295792    -0.292375  0.137186 -0.041077  …    -0.279410
2     0.825468  0.005417    -0.068235 -0.850737 -0.781911  …    -0.538637
3     0.458167  0.093774    -0.043214  0.476558  0.673355  …    -0.378961
4     0.183843 -0.736912     0.315946 -1.090088 -1.188561  …     0.247434

        p27  p27_pT157  p27_pT198  p38_MAPK  p38_pT180_Y182       p53  \
0 -0.158861  -0.128520  -0.277878  0.163695        0.978001 -0.131251
1  0.174650  -0.560634   0.226158 -0.234425        0.298065 -0.498628
2  1.388484   0.567361   0.277704 -0.680524       -0.651768  1.332241
3 -0.026071   0.002745  -0.004846 -0.244559       -0.085011  0.656739
4 -0.055098  -0.072531  -0.244982  0.034788        0.722223  0.034037
```

```
     p70S6K  p70S6K_pT389  p90RSK_pT359_S363
0  0.235811     -0.212356           0.210415
1  0.378535     -0.244212          -0.240162
2 -0.494482     -0.171739          -0.206528
3  0.296942     -0.245526           0.040049
4 -0.244347      0.363496           0.114600

[5 rows x 143 columns]
```

[4]:
```python
# intersect samples
set1 = set(protein_abundance["Sample"]).intersection(clinical["Sample"])

# subset dataframes
xclinical = clinical[clinical["Sample"].isin(set1)].copy()
xprotein = protein_abundance[protein_abundance["Sample"].isin(set1)].copy()

# keep only columns 1 and 9 (R is 1-based, Python is 0-based)
xclinical = xclinical.iloc[:, [0, 8]]

# remove rows that are neither "Positive" nor "Negative"
mask = xclinical["breast_carcinoma_estrogen_receptor_status"].isin(
    ["Positive", "Negative"]
)
xclinical = xclinical[mask]

# remove NA values
xclinical = xclinical.dropna(
    subset=["breast_carcinoma_estrogen_receptor_status"]
)

# merge clinical and protein data
mprotein = pd.merge(
    xclinical,
    xprotein,
    on="Sample",
    how="inner"
)
```

[5]:
```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# equivalent of complete.cases(t(mprotein))
sel = mprotein.notna().all(axis=1)

# set seed
np.random.seed(111)
```

```
# training indices: 2/3 of rows
n = mprotein.shape[0]
training_idx = np.random.choice(
    np.arange(n),
    size=int(2 * n / 3),
    replace=False
)

# split features (drop first two columns), selecciona las columnas 2: para␣
↪delante, es decir, desde la 3a columna.
# copy sirve para crear una copia, porque a veces lo que ghace es solo una vista
xtrain = mprotein.iloc[training_idx, 2:].copy()
xtest = mprotein.drop(training_idx).iloc[:, 2:].copy()

# scale features (IMPORTANT: fit on train, transform test)
scaler = StandardScaler()
xtrain = scaler.fit_transform(xtrain)
xtest = scaler.transform(xtest)

# extract labels (column 2 in R → index 1 in Python)
ytrain = mprotein.iloc[training_idx, 1]
ytest = mprotein.drop(training_idx).iloc[:, 1]

# encode labels
ylabels = np.where(ytrain == "Positive", 1, 0)
ytestlabels = np.where(ytest == "Positive", 1, 0)
```

2 Autoencoder

[6]:
```
from keras import Sequential

model = Sequential([
    layers.Dense(50, activation="relu", input_shape=(142,)),
    layers.Dense(20, activation="relu"),
    layers.Dense(50, activation="relu"),
    layers.Dense(142, activation="linear")
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:87:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1772527526.451746   54362 gpu_device.cc:2022] Created device
```

```
/job:localhost/replica:0/task:0/device:GPU:0 with 22314 MB memory:  -> device:
0, name: NVIDIA GeForce RTX 3090, pci bus id: 0000:01:00.0, compute capability:
8.6
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	7,150
dense_1 (Dense)	(None, 20)	1,020
dense_2 (Dense)	(None, 50)	1,050
dense_3 (Dense)	(None, 142)	7,242

Total params: 16,462 (64.30 KB)

Trainable params: 16,462 (64.30 KB)

Non-trainable params: 0 (0.00 B)

[7]:
```
model.compile(loss="mse",optimizer="rmsprop")
model.fit(xtrain,xtrain,epochs=25,batch_size=64,validation_split=0.2)
```

```
Epoch 1/25

WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1772527527.714473   54428 service.cc:148] XLA service 0x78c00c004660
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1772527527.714516   54428 service.cc:156]   StreamExecutor device
(0): NVIDIA GeForce RTX 3090, Compute Capability 8.6
2026-03-03 09:45:27.736008: I
tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1772527527.810825   54428 cuda_dnn.cc:529] Loaded cuDNN version
90800

1/4              9s 3s/step - loss:
1.1185

I0000 00:00:1772527530.410985   54428 device_compiler.h:188] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.
```

```
4/4              0s 564ms/step - loss:
1.0845

2026-03-03 09:45:33.077216: I
external/local_xla/xla/stream_executor/cuda/cuda_asm_compiler.cc:397] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_dot_38', 4 bytes spill stores, 4 bytes spill loads


4/4              6s 1s/step - loss:
1.0814 - val_loss: 0.9780
Epoch 2/25
4/4              4s 23ms/step - loss:
1.0228 - val_loss: 0.9602
Epoch 3/25
4/4              0s 28ms/step - loss:
0.9815 - val_loss: 0.9462
Epoch 4/25
4/4              0s 22ms/step - loss:
0.9689 - val_loss: 0.9316
Epoch 5/25
4/4              0s 30ms/step - loss:
0.9798 - val_loss: 0.9180
Epoch 6/25
4/4              0s 23ms/step - loss:
0.9751 - val_loss: 0.8970
Epoch 7/25
4/4              0s 26ms/step - loss:
0.8915 - val_loss: 0.8750
Epoch 8/25
4/4              0s 35ms/step - loss:
0.9167 - val_loss: 0.8499
Epoch 9/25
4/4              0s 24ms/step - loss:
0.8821 - val_loss: 0.8159
Epoch 10/25
4/4              0s 22ms/step - loss:
0.8381 - val_loss: 0.7910
Epoch 11/25
4/4              0s 21ms/step - loss:
0.8160 - val_loss: 0.7664
Epoch 12/25
4/4              0s 28ms/step - loss:
0.7708 - val_loss: 0.7441
Epoch 13/25
4/4              0s 27ms/step - loss:
0.7457 - val_loss: 0.7170
Epoch 14/25
4/4              0s 37ms/step - loss:
```

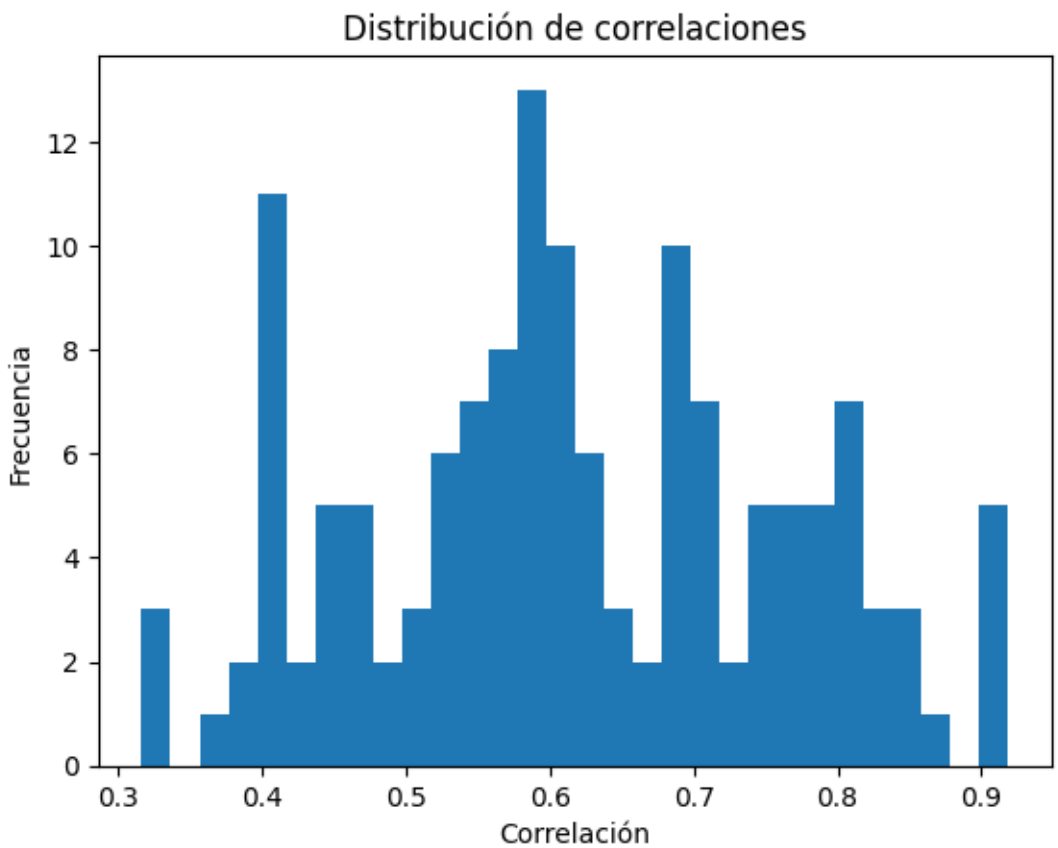
```
0.7250 - val_loss: 0.7019
Epoch 15/25
4/4              0s 25ms/step - loss:
0.7163 - val_loss: 0.6889
Epoch 16/25
4/4              0s 26ms/step - loss:
0.7097 - val_loss: 0.6799
Epoch 17/25
4/4              0s 32ms/step - loss:
0.6942 - val_loss: 0.6705
Epoch 18/25
4/4              0s 23ms/step - loss:
0.6708 - val_loss: 0.6622
Epoch 19/25
4/4              0s 27ms/step - loss:
0.6441 - val_loss: 0.6554
Epoch 20/25
4/4              0s 25ms/step - loss:
0.6499 - val_loss: 0.6479
Epoch 21/25
4/4              0s 29ms/step - loss:
0.6457 - val_loss: 0.6430
Epoch 22/25
4/4              0s 29ms/step - loss:
0.6338 - val_loss: 0.6375
Epoch 23/25
4/4              0s 28ms/step - loss:
0.6301 - val_loss: 0.6319
Epoch 24/25
4/4              0s 27ms/step - loss:
0.6156 - val_loss: 0.6279
Epoch 25/25
4/4              0s 26ms/step - loss:
0.6257 - val_loss: 0.6249
```

[7]:
```
<keras.src.callbacks.history.History at 0x78c18cac4ef0>
```

[8]:
```python
import matplotlib.pyplot as plt

# predicción del modelo (x.hat)
x_hat = model.predict(xtrain)

# correlación columna a columna
corr_matrix = np.corrcoef(x_hat.T, xtrain.T)

# extraer la diagonal (correlación de cada proteína consigo misma)
n_features = xtrain.shape[1]
```

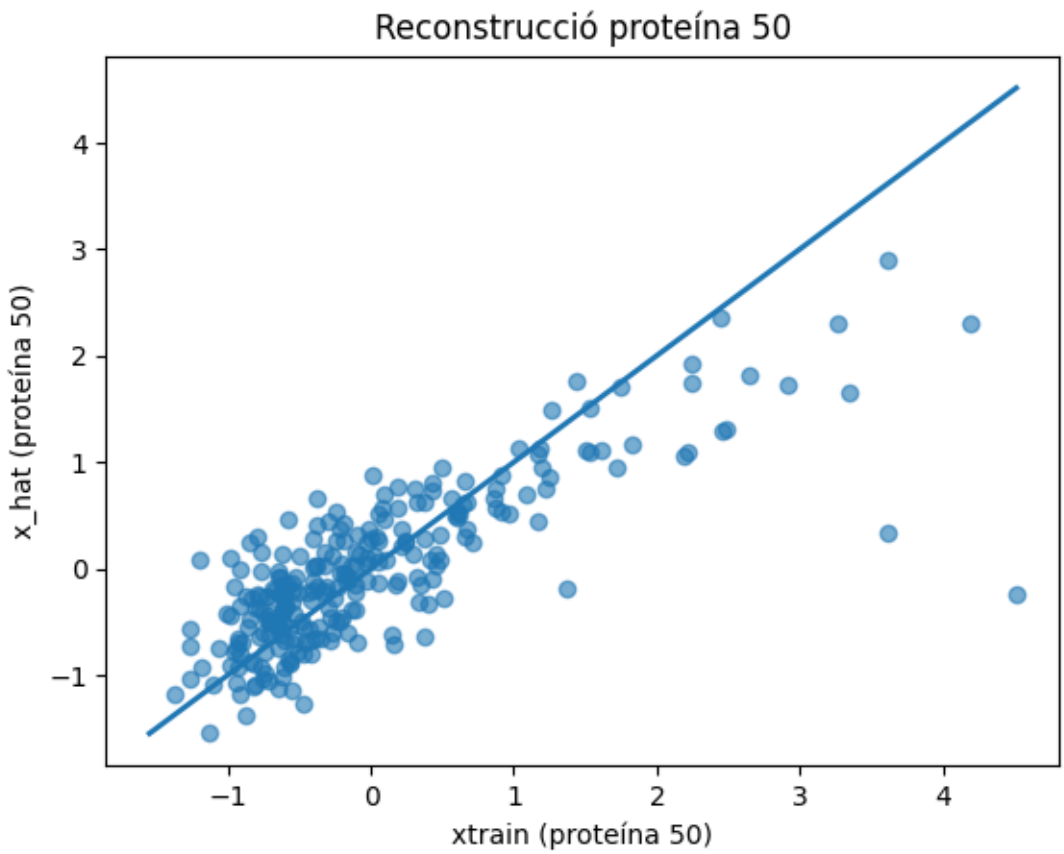
```
vcor = np.diag(corr_matrix[:n_features, n_features:])

# histograma
plt.hist(vcor, bins=30)
plt.xlabel("Correlación")
plt.ylabel("Frecuencia")
plt.title("Distribución de correlaciones")
plt.show()
```

```
8/8              2s 131ms/step
```



[9]:
```
# scatter plot: reconstruido vs original (proteína 50)
plt.scatter(xtrain[:, 49], x_hat[:, 49], alpha=0.6) # alpha es la trasnparencia␣
↪del punts.

# recta y = x
min_val = min(xtrain[:, 49].min(), x_hat[:, 49].min())
max_val = max(xtrain[:, 49].max(), x_hat[:, 49].max())
plt.plot([min_val, max_val], [min_val, max_val], linewidth=2)
```

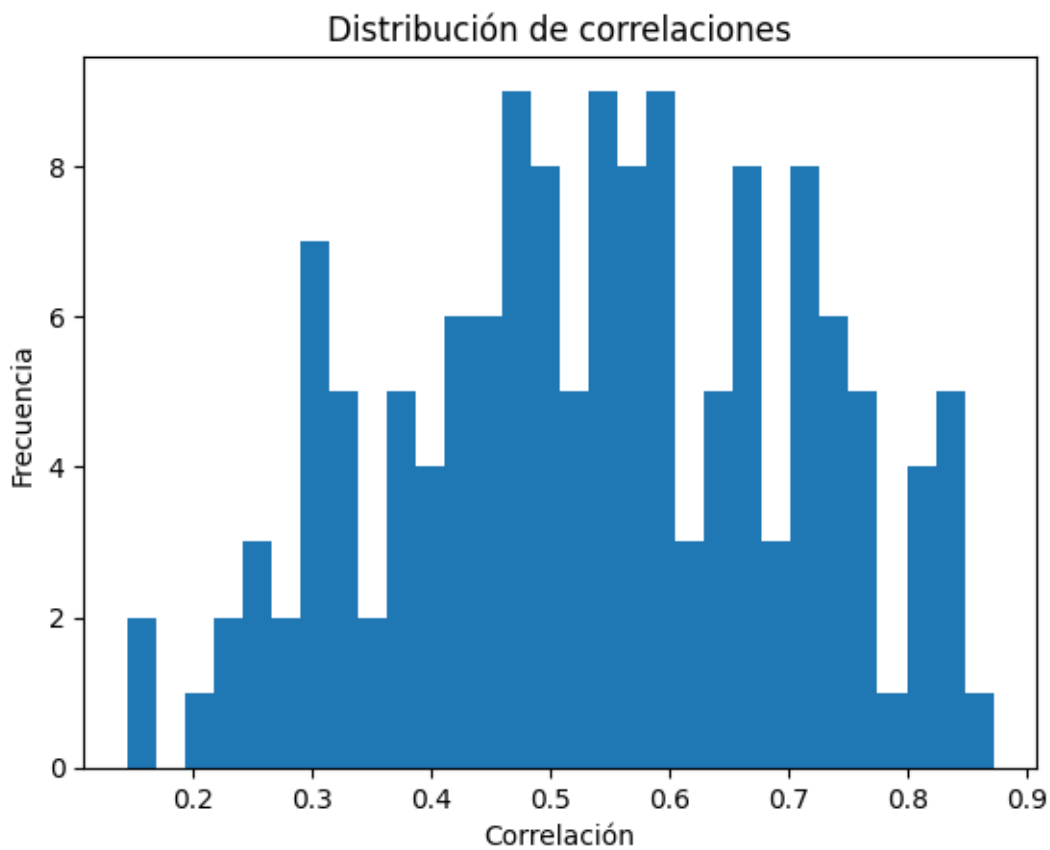
```
plt.xlabel("xtrain (proteína 50)")
plt.ylabel("x_hat (proteína 50)")
plt.title("Reconstrucció proteína 50")
plt.show()
```

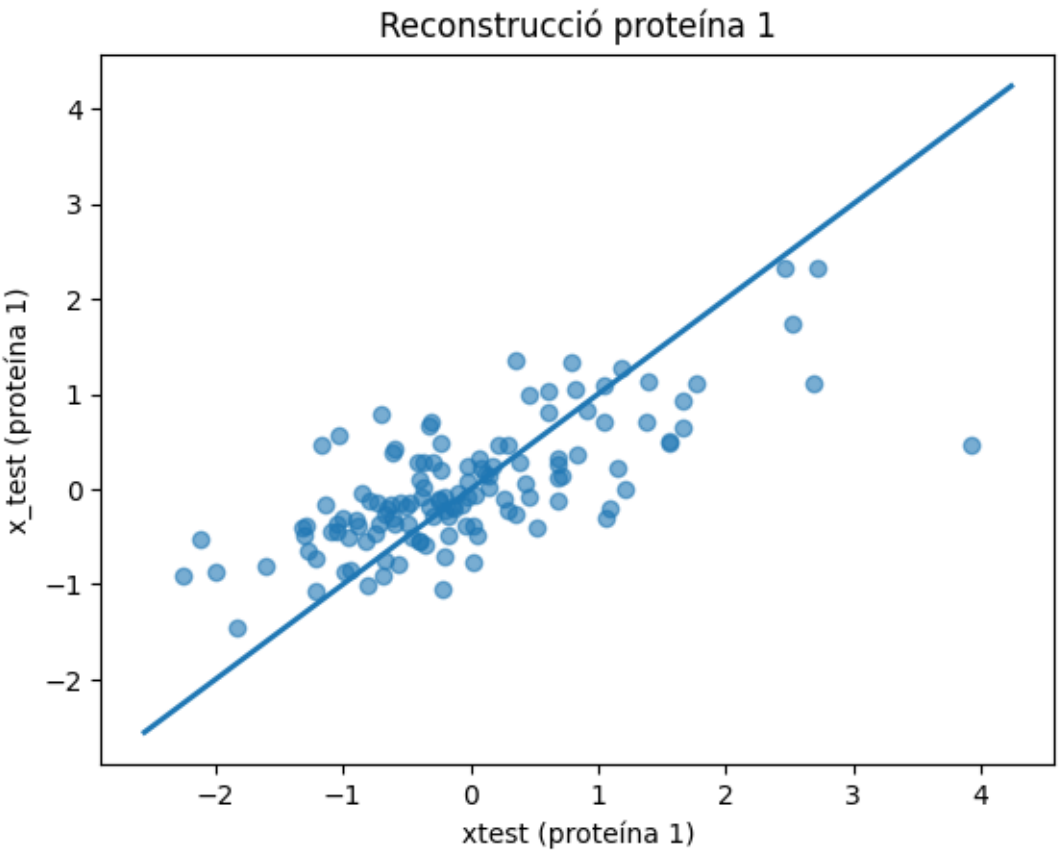


[10]:
```
# predicción del modelo (x.hat)
x_hat = model.predict(xtest)

# correlación columna a columna
corr_matrix = np.corrcoef(x_hat.T, xtest.T) # la T es trasposar la matriu,␣
↪numpy asumeix que les files son les variables

# extraer la diagonal (correlación de cada proteína consigo misma)
n_features = xtest.shape[1]
vcor = np.diag(corr_matrix[:n_features, n_features:])

# histograma
plt.hist(vcor, bins=30)
plt.xlabel("Correlación")
```

```
plt.ylabel("Frecuencia")
plt.title("Distribución de correlaciones")
plt.show()

# scatter plot: reconstruido vs original (proteína 50)
plt.scatter(xtest[:, 0], x_hat[:, 0], alpha=0.6) # alpha es la trasnparencia␣
 ↪del punts.

# recta y = x
plt.plot(plt.xlim(), plt.xlim(), linewidth=2)

plt.xlabel("xtest (proteína 1)")
plt.ylabel("x_test (proteína 1)")
plt.title("Reconstrucció proteína 1")
plt.show()
```

```
4/4              0s 6ms/step
```





3 Defining Encoder and Decoder submodels

[11]:
```
# capa de entrada encoder
input_enc = keras.Input(shape=(142,))

# capas del encoder
encoded = layers.Dense(50, activation="relu")(input_enc)
output_enc = layers.Dense(20, activation="relu")(encoded)

# modelo encoder
encoder = keras.Model(inputs=input_enc, outputs=output_enc)

# resumen del modelo
encoder.summary()

# capa entrada decoder
input_dec = keras.Input(shape=(20,))

# capas del decoder
```

```
decoded = layers.Dense(50, activation="relu")(input_dec)
output_dec = layers.Dense(142, activation="linear")(decoded)

# modelo decoder

decoder= keras.Model(inputs=input_dec, outputs=output_dec)

# resumen del modelo
decoder.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 142)	0
dense_4 (Dense)	(None, 50)	7,150
dense_5 (Dense)	(None, 20)	1,020

Total params: 8,170 (31.91 KB)

Trainable params: 8,170 (31.91 KB)

Non-trainable params: 0 (0.00 B)

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 20)	0
dense_6 (Dense)	(None, 50)	1,050
dense_7 (Dense)	(None, 142)	7,242

Total params: 8,292 (32.39 KB)

Trainable params: 8,292 (32.39 KB)

Non-trainable params: 0 (0.00 B)

4 Defining Autoencoder

[12]:
```
aen = keras.Model(inputs=input_enc, outputs=decoder(encoder(input_enc)))
aen.summary()
```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 142)	0
functional_1 (Functional)	(None, 20)	8,170
functional_2 (Functional)	(None, 142)	8,292

Total params: 16,462 (64.30 KB)

Trainable params: 16,462 (64.30 KB)

Non-trainable params: 0 (0.00 B)

[13]:
```
aen.compile(optimizer="rmsprop", loss="mse")
aen.fit(xtrain, xtrain, epochs=25, batch_size=64, validation_split=0.2)
```

```
Epoch 1/25
4/4              3s 367ms/step - loss:
1.0400 - val_loss: 0.9730
Epoch 2/25
4/4              1s 26ms/step - loss:
1.0387 - val_loss: 0.9629
Epoch 3/25
4/4              0s 23ms/step - loss:
1.0051 - val_loss: 0.9518
Epoch 4/25
4/4              0s 23ms/step - loss:
1.0005 - val_loss: 0.9410
Epoch 5/25
4/4              0s 26ms/step - loss:
0.9756 - val_loss: 0.9272
Epoch 6/25
```

```
4/4              0s 24ms/step - loss:
0.9658 - val_loss: 0.9102
Epoch 7/25
4/4              0s 28ms/step - loss:
0.9381 - val_loss: 0.8946
Epoch 8/25
4/4              0s 19ms/step - loss:
0.9110 - val_loss: 0.8714
Epoch 9/25
4/4              0s 26ms/step - loss:
0.8792 - val_loss: 0.8526
Epoch 10/25
4/4              0s 24ms/step - loss:
0.8709 - val_loss: 0.8295
Epoch 11/25
4/4              0s 36ms/step - loss:
0.8411 - val_loss: 0.8110
Epoch 12/25
4/4              0s 26ms/step - loss:
0.8104 - val_loss: 0.7913
Epoch 13/25
4/4              0s 28ms/step - loss:
0.7991 - val_loss: 0.7717
Epoch 14/25
4/4              0s 25ms/step - loss:
0.7851 - val_loss: 0.7550
Epoch 15/25
4/4              0s 23ms/step - loss:
0.7512 - val_loss: 0.7360
Epoch 16/25
4/4              0s 25ms/step - loss:
0.7445 - val_loss: 0.7180
Epoch 17/25
4/4              0s 29ms/step - loss:
0.7135 - val_loss: 0.7035
Epoch 18/25
4/4              0s 27ms/step - loss:
0.7005 - val_loss: 0.6871
Epoch 19/25
4/4              0s 22ms/step - loss:
0.6897 - val_loss: 0.6755
Epoch 20/25
4/4              0s 29ms/step - loss:
0.6743 - val_loss: 0.6643
Epoch 21/25
4/4              0s 22ms/step - loss:
0.6705 - val_loss: 0.6548
Epoch 22/25
```

```
4/4              0s 22ms/step - loss:
0.6420 - val_loss: 0.6470
Epoch 23/25
4/4              0s 22ms/step - loss:
0.6463 - val_loss: 0.6390
Epoch 24/25
4/4              0s 24ms/step - loss:
0.6394 - val_loss: 0.6328
Epoch 25/25
4/4              0s 22ms/step - loss:
0.6324 - val_loss: 0.6265
```

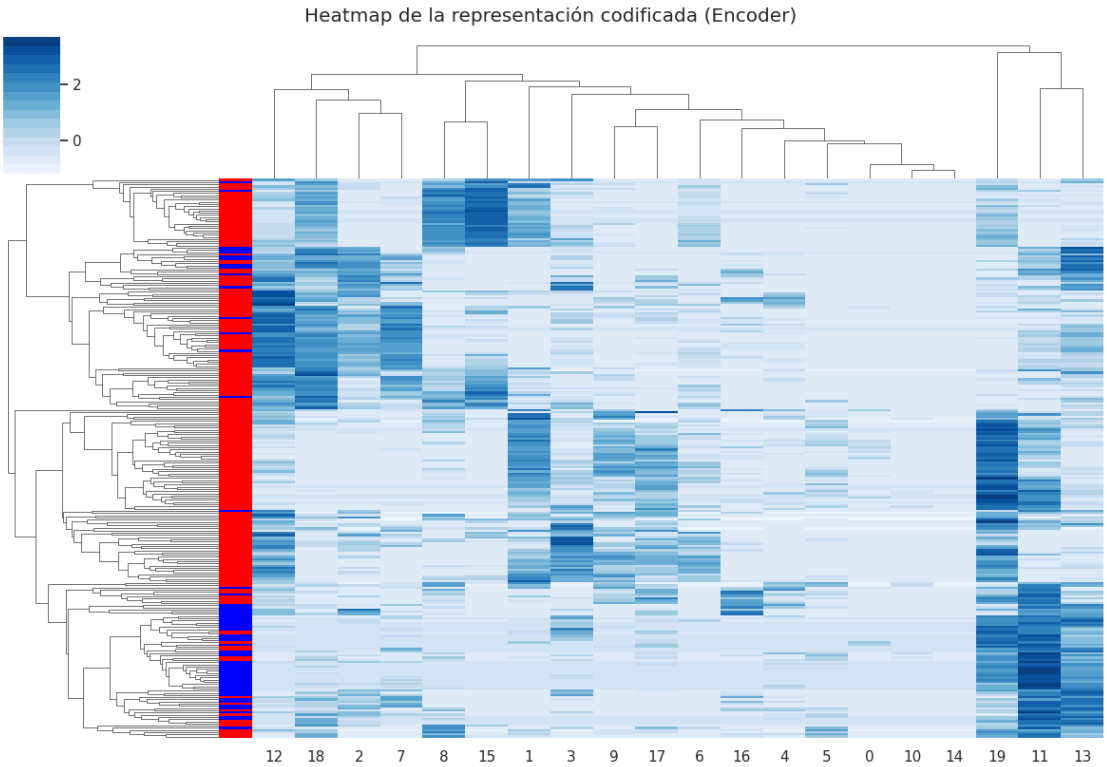
[13]:
```
<keras.src.callbacks.history.History at 0x78c189b64d70>
```

[14]:
```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# Generar representaciones codificadas y decodificadas
encoded_expression = encoder.predict(xtrain)
decoded_expression = decoder.predict(encoded_expression)

# Escalar por fila (equivalente a scale="row" en R)
scaler = StandardScaler(with_mean=True, with_std=True)
encoded_scaled = scaler.fit_transform(encoded_expression.T).T

# Paleta de colores similar a brewer.pal(8, "Blues") con 15 niveles
col_main = sns.color_palette("Blues", 15)
```

```
8/8              0s 27ms/step
8/8              0s 24ms/step
```

[15]:
```python
# Colores de filas según etiquetas ----
# Suponiendo ylabels = 0/1 (0=Negativo, 1=Positivo)
row_colors = np.array(['red' if y == 1 else 'blue' for y in ylabels])

sns.set(style="white")
g = sns.clustermap(
    encoded_scaled,           # datos normalizados
    row_colors=row_colors,    # colores de filas
    col_cluster=True,
    cmap=col_main,             # paleta azul
    yticklabels=False,
    xticklabels=True,
    figsize=(12, 8)
)
```

```
plt.suptitle("Heatmap de la representación codificada (Encoder)", y=1.02)
plt.show()
```



5 Stacked autoencoder

[16]:
```
#AE1
input_enc1 = keras.Input(shape=(142,))
output_enc1 = layers.Dense(50, activation="relu")(input_enc1)
encoder1 = keras.Model(inputs=input_enc1, outputs=output_enc1)
encoder1.summary()
input_dec1 = keras.Input(shape=(50,))
output_dec1 = layers.Dense(142, activation="linear")(input_dec1)
decoder1 = keras.Model(inputs=input_dec1, outputs=output_dec1)
decoder1.summary()

aen_input1 = keras.Input(shape=(142,))
aen_output1 = decoder1(encoder1(aen_input1))
sae1 = keras.Model(inputs=aen_input1, outputs=aen_output1)
sae1.summary()

sae1.compile(optimizer="rmsprop", loss="mse")
sae1.fit(xtrain, xtrain, epochs=25, batch_size=64, validation_split=0.2)
```

```
encoded_expression1 = encoder1.predict(xtrain)


#AE2
input_enc2 = keras.Input(shape=(50,))
output_enc2 = layers.Dense(20, activation="relu")(input_enc2)
encoder2 = keras.Model(inputs=input_enc2, outputs=output_enc2)
encoder2.summary()
input_dec2 = keras.Input(shape=(20,))
output_dec2 = layers.Dense(50, activation="linear")(input_dec2)
decoder2 = keras.Model(inputs=input_dec2, outputs=output_dec2)
decoder2.summary()

aen_input2 = keras.Input(shape=(50,))
aen_output2 = decoder2(encoder2(aen_input2))
sae2 = keras.Model(inputs=aen_input2, outputs=aen_output2)
sae2.summary()

sae2.compile(optimizer="rmsprop", loss="mse")
sae2.fit(encoded_expression1, encoded_expression1, epochs=25, batch_size=64,␣
 ↪validation_split=0.2)
encoded_expression2 = encoder2.predict(encoded_expression1)

#AE3
input_enc3 = keras.Input(shape=(20,))
output_enc3 = layers.Dense(10, activation="relu")(input_enc3)
encoder3 = keras.Model(inputs=input_enc3, outputs=output_enc3)
encoder3.summary()
input_dec3 = keras.Input(shape=(10,))
output_dec3 = layers.Dense(20, activation="linear")(input_dec3)
decoder3 = keras.Model(inputs=input_dec3, outputs=output_dec3)
decoder3.summary()

aen_input3 = keras.Input(shape=(20,))
aen_output3 = decoder3(encoder3(aen_input3))
sae3 = keras.Model(inputs=aen_input3, outputs=aen_output3)
sae3.summary()

sae3.compile(optimizer="rmsprop", loss="mse")
sae3.fit(encoded_expression2, encoded_expression2, epochs=25, batch_size=64,␣
 ↪validation_split=0.2)
encoded_expression3 = encoder3.predict(encoded_expression2)
```

```
Model: "functional_4"


 Layer (type)                    Output Shape                  Param #
```

```
 input_layer_3 (InputLayer)        (None, 142)                        0

 dense_8 (Dense)                   (None, 50)                     7,150

Total params: 7,150 (27.93 KB)

Trainable params: 7,150 (27.93 KB)

Non-trainable params: 0 (0.00 B)

Model: "functional_5"

 Layer (type)                      Output Shape                   Param #

 input_layer_4 (InputLayer)        (None, 50)                         0

 dense_9 (Dense)                   (None, 142)                    7,242

Total params: 7,242 (28.29 KB)

Trainable params: 7,242 (28.29 KB)

Non-trainable params: 0 (0.00 B)

Model: "functional_6"

 Layer (type)                      Output Shape                   Param #

 input_layer_5 (InputLayer)        (None, 142)                        0

 functional_4 (Functional)         (None, 50)                     7,150

 functional_5 (Functional)         (None, 142)                    7,242

Total params: 14,392 (56.22 KB)
```

```
 Trainable params: 14,392 (56.22 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/25
4/4              1s 224ms/step - loss:
1.3134 - val_loss: 1.1393
Epoch 2/25
4/4              0s 34ms/step - loss:
1.1892 - val_loss: 1.0743
Epoch 3/25
4/4              0s 23ms/step - loss:
1.1030 - val_loss: 1.0252
Epoch 4/25
4/4              0s 34ms/step - loss:
1.0039 - val_loss: 0.9846
Epoch 5/25
4/4              0s 26ms/step - loss:
0.9964 - val_loss: 0.9427
Epoch 6/25
4/4              0s 35ms/step - loss:
0.9333 - val_loss: 0.9083
Epoch 7/25
4/4              0s 28ms/step - loss:
0.9024 - val_loss: 0.8708
Epoch 8/25
4/4              0s 25ms/step - loss:
0.8862 - val_loss: 0.8406
Epoch 9/25
4/4              0s 22ms/step - loss:
0.8347 - val_loss: 0.8107
Epoch 10/25
4/4              0s 24ms/step - loss:
0.7992 - val_loss: 0.7857
Epoch 11/25
4/4              0s 23ms/step - loss:
0.7591 - val_loss: 0.7602
Epoch 12/25
4/4              0s 20ms/step - loss:
0.7439 - val_loss: 0.7386
Epoch 13/25
4/4              0s 17ms/step - loss:
0.7134 - val_loss: 0.7227
Epoch 14/25
4/4              0s 25ms/step - loss:
0.6863 - val_loss: 0.7074
Epoch 15/25
```

```
4/4              0s 30ms/step - loss:
0.6594 - val_loss: 0.6933
Epoch 16/25
4/4              0s 27ms/step - loss:
0.6591 - val_loss: 0.6807
Epoch 17/25
4/4              0s 25ms/step - loss:
0.6340 - val_loss: 0.6682
Epoch 18/25
4/4              0s 22ms/step - loss:
0.6307 - val_loss: 0.6560
Epoch 19/25
4/4              0s 25ms/step - loss:
0.6080 - val_loss: 0.6455
Epoch 20/25
4/4              0s 24ms/step - loss:
0.5903 - val_loss: 0.6357
Epoch 21/25
4/4              0s 27ms/step - loss:
0.5887 - val_loss: 0.6276
Epoch 22/25
4/4              0s 26ms/step - loss:
0.5766 - val_loss: 0.6172
Epoch 23/25
4/4              0s 22ms/step - loss:
0.5652 - val_loss: 0.6093
Epoch 24/25
4/4              0s 28ms/step - loss:
0.5426 - val_loss: 0.6021
Epoch 25/25
4/4              0s 25ms/step - loss:
0.5410 - val_loss: 0.5915
WARNING:tensorflow:5 out of the last 17 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x78c18c89bba0> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.
8/8              0s 14ms/step
```

Model: "functional_7"

Layer (type)	Output Shape	Param #

```
 input_layer_6 (InputLayer)        (None, 50)                          0
 dense_10 (Dense)                  (None, 20)                      1,020

Total params: 1,020 (3.98 KB)

Trainable params: 1,020 (3.98 KB)

Non-trainable params: 0 (0.00 B)

Model: "functional_8"

 Layer (type)                      Output Shape                   Param #
 input_layer_7 (InputLayer)        (None, 20)                          0
 dense_11 (Dense)                  (None, 50)                      1,050

Total params: 1,050 (4.10 KB)

Trainable params: 1,050 (4.10 KB)

Non-trainable params: 0 (0.00 B)

Model: "functional_9"

 Layer (type)                      Output Shape                   Param #
 input_layer_8 (InputLayer)        (None, 50)                          0
 functional_7 (Functional)         (None, 20)                      1,020
 functional_8 (Functional)         (None, 50)                      1,050

Total params: 2,070 (8.09 KB)
```

```
 Trainable params: 2,070 (8.09 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/25
4/4              2s 320ms/step - loss:
1.9795 - val_loss: 1.4649
Epoch 2/25
4/4              0s 27ms/step - loss:
1.5418 - val_loss: 1.3210
Epoch 3/25
4/4              0s 25ms/step - loss:
1.4111 - val_loss: 1.2335
Epoch 4/25
4/4              0s 34ms/step - loss:
1.2583 - val_loss: 1.1494
Epoch 5/25
4/4              0s 26ms/step - loss:
1.2172 - val_loss: 1.0802
Epoch 6/25
4/4              0s 37ms/step - loss:
1.1099 - val_loss: 1.0354
Epoch 7/25
4/4              0s 30ms/step - loss:
1.0913 - val_loss: 0.9955
Epoch 8/25
4/4              0s 23ms/step - loss:
1.0204 - val_loss: 0.9508
Epoch 9/25
4/4              0s 28ms/step - loss:
0.9344 - val_loss: 0.9060
Epoch 10/25
4/4              0s 23ms/step - loss:
0.9290 - val_loss: 0.8775
Epoch 11/25
4/4              0s 23ms/step - loss:
0.8980 - val_loss: 0.8476
Epoch 12/25
4/4              0s 26ms/step - loss:
0.8767 - val_loss: 0.8221
Epoch 13/25
4/4              0s 37ms/step - loss:
0.8390 - val_loss: 0.7989
Epoch 14/25
4/4              0s 25ms/step - loss:
0.8252 - val_loss: 0.7756
Epoch 15/25
```

```
4/4              0s 31ms/step - loss:
0.7328 - val_loss: 0.7521
Epoch 16/25
4/4              0s 26ms/step - loss:
0.7071 - val_loss: 0.7297
Epoch 17/25
4/4              0s 28ms/step - loss:
0.7126 - val_loss: 0.7096
Epoch 18/25
4/4              0s 23ms/step - loss:
0.7035 - val_loss: 0.6878
Epoch 19/25
4/4              0s 27ms/step - loss:
0.6966 - val_loss: 0.6709
Epoch 20/25
4/4              0s 27ms/step - loss:
0.6533 - val_loss: 0.6517
Epoch 21/25
4/4              0s 27ms/step - loss:
0.6334 - val_loss: 0.6322
Epoch 22/25
4/4              0s 26ms/step - loss:
0.6196 - val_loss: 0.6163
Epoch 23/25
4/4              0s 24ms/step - loss:
0.6024 - val_loss: 0.5954
Epoch 24/25
4/4              0s 22ms/step - loss:
0.5851 - val_loss: 0.5787
Epoch 25/25
4/4              0s 26ms/step - loss:
0.5740 - val_loss: 0.5649
WARNING:tensorflow:5 out of the last 17 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
0x78c1881c3a60> triggered tf.function retracing. Tracing is expensive and the
excessive number of tracings could be due to (1) creating @tf.function
repeatedly in a loop, (2) passing tensors with different shapes, (3) passing
Python objects instead of tensors. For (1), please define your @tf.function
outside of the loop. For (2), @tf.function has reduce_retracing=True option that
can avoid unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.
8/8              0s 13ms/step
```

Model: "functional_10"

Layer (type)	Output Shape	Param #

```
 input_layer_9 (InputLayer)        (None, 20)                          0
 dense_12 (Dense)                  (None, 10)                        210

Total params: 210 (840.00 B)

Trainable params: 210 (840.00 B)

Non-trainable params: 0 (0.00 B)

Model: "functional_11"

 Layer (type)                      Output Shape                   Param #
 input_layer_10 (InputLayer)       (None, 10)                          0
 dense_13 (Dense)                  (None, 20)                        220

Total params: 220 (880.00 B)

Trainable params: 220 (880.00 B)

Non-trainable params: 0 (0.00 B)

Model: "functional_12"

 Layer (type)                      Output Shape                   Param #
 input_layer_11 (InputLayer)       (None, 20)                          0
 functional_10 (Functional)        (None, 10)                        210
 functional_11 (Functional)        (None, 20)                        220

Total params: 430 (1.68 KB)
```

```
 Trainable params: 430 (1.68 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/25
4/4              2s 301ms/step - loss:
1.9190 - val_loss: 1.5840
Epoch 2/25
4/4              0s 27ms/step - loss:
1.7593 - val_loss: 1.4951
Epoch 3/25
4/4              0s 26ms/step - loss:
1.6345 - val_loss: 1.4272
Epoch 4/25
4/4              0s 20ms/step - loss:
1.5526 - val_loss: 1.3752
Epoch 5/25
4/4              0s 26ms/step - loss:
1.4482 - val_loss: 1.3263
Epoch 6/25
4/4              0s 23ms/step - loss:
1.4487 - val_loss: 1.2838
Epoch 7/25
4/4              0s 25ms/step - loss:
1.4422 - val_loss: 1.2468
Epoch 8/25
4/4              0s 28ms/step - loss:
1.3645 - val_loss: 1.2095
Epoch 9/25
4/4              0s 25ms/step - loss:
1.4171 - val_loss: 1.1726
Epoch 10/25
4/4              0s 27ms/step - loss:
1.3272 - val_loss: 1.1421
Epoch 11/25
4/4              0s 27ms/step - loss:
1.2760 - val_loss: 1.1119
Epoch 12/25
4/4              0s 27ms/step - loss:
1.2309 - val_loss: 1.0828
Epoch 13/25
4/4              0s 23ms/step - loss:
1.2502 - val_loss: 1.0557
Epoch 14/25
4/4              0s 24ms/step - loss:
1.2015 - val_loss: 1.0293
Epoch 15/25
```

```
4/4              0s 26ms/step - loss:
1.1651 - val_loss: 1.0062
Epoch 16/25
4/4              0s 26ms/step - loss:
1.0663 - val_loss: 0.9830
Epoch 17/25
4/4              0s 29ms/step - loss:
1.0213 - val_loss: 0.9631
Epoch 18/25
4/4              0s 29ms/step - loss:
1.0088 - val_loss: 0.9427
Epoch 19/25
4/4              0s 26ms/step - loss:
1.0627 - val_loss: 0.9220
Epoch 20/25
4/4              0s 30ms/step - loss:
0.9921 - val_loss: 0.9041
Epoch 21/25
4/4              0s 25ms/step - loss:
1.0227 - val_loss: 0.8877
Epoch 22/25
4/4              0s 23ms/step - loss:
0.9540 - val_loss: 0.8708
Epoch 23/25
4/4              0s 23ms/step - loss:
0.9770 - val_loss: 0.8539
Epoch 24/25
4/4              0s 25ms/step - loss:
0.9514 - val_loss: 0.8375
Epoch 25/25
4/4              0s 30ms/step - loss:
0.9412 - val_loss: 0.8212
8/8              0s 11ms/step
```

[17]:
```python
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
# Final Model
sae_input = keras.Input(shape=(142,))
sae_output = encoder3(encoder2(encoder1(sae_input)))
sae_output = layers.Dense(5,activation="relu")(sae_output)
sae_output = layers.Dense(1,activation="sigmoid")(sae_output)
sae = keras.Model(inputs=sae_input, outputs=sae_output)
sae.summary()
```

```
Model: "functional_13"
```

Layer (type)	Output Shape	Param #

```
 input_layer_12 (InputLayer)      (None, 142)                        0

 functional_4 (Functional)        (None, 50)                     7,150

 functional_7 (Functional)        (None, 20)                     1,020

 functional_10 (Functional)       (None, 10)                       210

 dense_14 (Dense)                 (None, 5)                         55

 dense_15 (Dense)                 (None, 1)                          6


 Total params: 8,441 (32.97 KB)

 Trainable params: 8,441 (32.97 KB)

 Non-trainable params: 0 (0.00 B)
```

[18]:
```python
for layer in sae.layers[0:3]:
    layer.trainable = False
sae.summary()
for layer in sae.layers:
    print(layer.name, layer.trainable)
    # se coongelan los encoders, participan pero no se ajusta.
```

```
Model: "functional_13"


 Layer (type)                     Output Shape                  Param #

 input_layer_12 (InputLayer)      (None, 142)                        0

 functional_4 (Functional)        (None, 50)                     7,150

 functional_7 (Functional)        (None, 20)                     1,020

 functional_10 (Functional)       (None, 10)                       210

 dense_14 (Dense)                 (None, 5)                         55

 dense_15 (Dense)                 (None, 1)                          6
```

```
 Total params: 8,441 (32.97 KB)

 Trainable params: 271 (1.06 KB)

 Non-trainable params: 8,170 (31.91 KB)

input_layer_12 False
functional_4 False
functional_7 False
functional_10 True
dense_14 True
dense_15 True
```

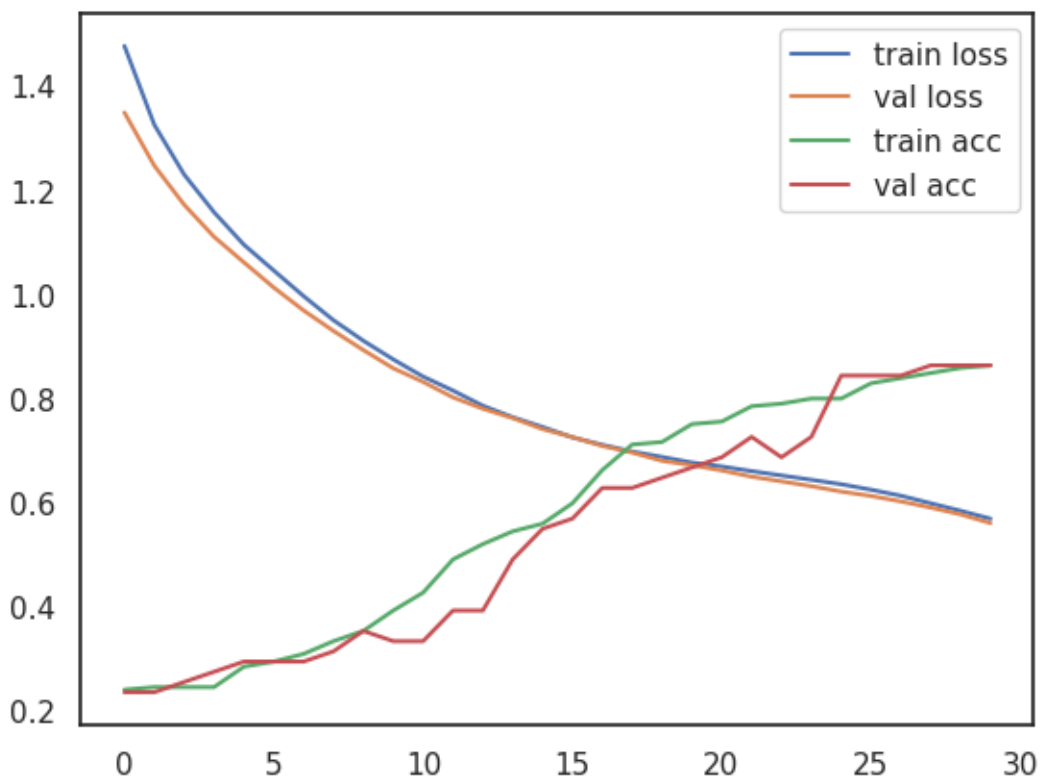
[19]:
```
sae.compile(optimizer="rmsprop", loss="binary_crossentropy",␣
 ↪metrics=["accuracy"])
history = sae.fit(xtrain, ylabels,epochs=30,batch_size=64,validation_split=0.2)
```

```
Epoch 1/30
4/4              3s 471ms/step -
accuracy: 0.2190 - loss: 1.5256 - val_accuracy: 0.2353 - val_loss: 1.3481
Epoch 2/30
4/4              0s 25ms/step -
accuracy: 0.2428 - loss: 1.3264 - val_accuracy: 0.2353 - val_loss: 1.2461
Epoch 3/30
4/4              0s 27ms/step -
accuracy: 0.2454 - loss: 1.2221 - val_accuracy: 0.2549 - val_loss: 1.1711
Epoch 4/30
4/4              0s 23ms/step -
accuracy: 0.2434 - loss: 1.1820 - val_accuracy: 0.2745 - val_loss: 1.1096
Epoch 5/30
4/4              0s 25ms/step -
accuracy: 0.2965 - loss: 1.0716 - val_accuracy: 0.2941 - val_loss: 1.0605
Epoch 6/30
4/4              0s 30ms/step -
accuracy: 0.2807 - loss: 1.0584 - val_accuracy: 0.2941 - val_loss: 1.0122
Epoch 7/30
4/4              0s 26ms/step -
accuracy: 0.3152 - loss: 0.9953 - val_accuracy: 0.2941 - val_loss: 0.9681
Epoch 8/30
4/4              0s 30ms/step -
accuracy: 0.3318 - loss: 0.9499 - val_accuracy: 0.3137 - val_loss: 0.9288
Epoch 9/30
4/4              0s 30ms/step -
accuracy: 0.3443 - loss: 0.9201 - val_accuracy: 0.3529 - val_loss: 0.8921
Epoch 10/30
4/4              0s 37ms/step -
```

```
accuracy: 0.4048 - loss: 0.8588 - val_accuracy: 0.3333 - val_loss: 0.8572
Epoch 11/30
4/4            0s 23ms/step -
accuracy: 0.3899 - loss: 0.8807 - val_accuracy: 0.3333 - val_loss: 0.8311
Epoch 12/30
4/4            0s 26ms/step -
accuracy: 0.4846 - loss: 0.8240 - val_accuracy: 0.3922 - val_loss: 0.8014
Epoch 13/30
4/4            0s 27ms/step -
accuracy: 0.4813 - loss: 0.7961 - val_accuracy: 0.3922 - val_loss: 0.7795
Epoch 14/30
4/4            0s 31ms/step -
accuracy: 0.5479 - loss: 0.7663 - val_accuracy: 0.4902 - val_loss: 0.7616
Epoch 15/30
4/4            0s 23ms/step -
accuracy: 0.5558 - loss: 0.7405 - val_accuracy: 0.5490 - val_loss: 0.7404
Epoch 16/30
4/4            0s 34ms/step -
accuracy: 0.5929 - loss: 0.7257 - val_accuracy: 0.5686 - val_loss: 0.7251
Epoch 17/30
4/4            0s 30ms/step -
accuracy: 0.6569 - loss: 0.7110 - val_accuracy: 0.6275 - val_loss: 0.7081
Epoch 18/30
4/4            0s 31ms/step -
accuracy: 0.6999 - loss: 0.6981 - val_accuracy: 0.6275 - val_loss: 0.6953
Epoch 19/30
4/4            0s 27ms/step -
accuracy: 0.7285 - loss: 0.6847 - val_accuracy: 0.6471 - val_loss: 0.6795
Epoch 20/30
4/4            0s 23ms/step -
accuracy: 0.7578 - loss: 0.6773 - val_accuracy: 0.6667 - val_loss: 0.6711
Epoch 21/30
4/4            0s 27ms/step -
accuracy: 0.7494 - loss: 0.6720 - val_accuracy: 0.6863 - val_loss: 0.6610
Epoch 22/30
4/4            0s 25ms/step -
accuracy: 0.7950 - loss: 0.6623 - val_accuracy: 0.7255 - val_loss: 0.6490
Epoch 23/30
4/4            0s 31ms/step -
accuracy: 0.7777 - loss: 0.6555 - val_accuracy: 0.6863 - val_loss: 0.6401
Epoch 24/30
4/4            0s 29ms/step -
accuracy: 0.8076 - loss: 0.6403 - val_accuracy: 0.7255 - val_loss: 0.6308
Epoch 25/30
4/4            0s 29ms/step -
accuracy: 0.7941 - loss: 0.6323 - val_accuracy: 0.8431 - val_loss: 0.6205
Epoch 26/30
4/4            0s 30ms/step -
```

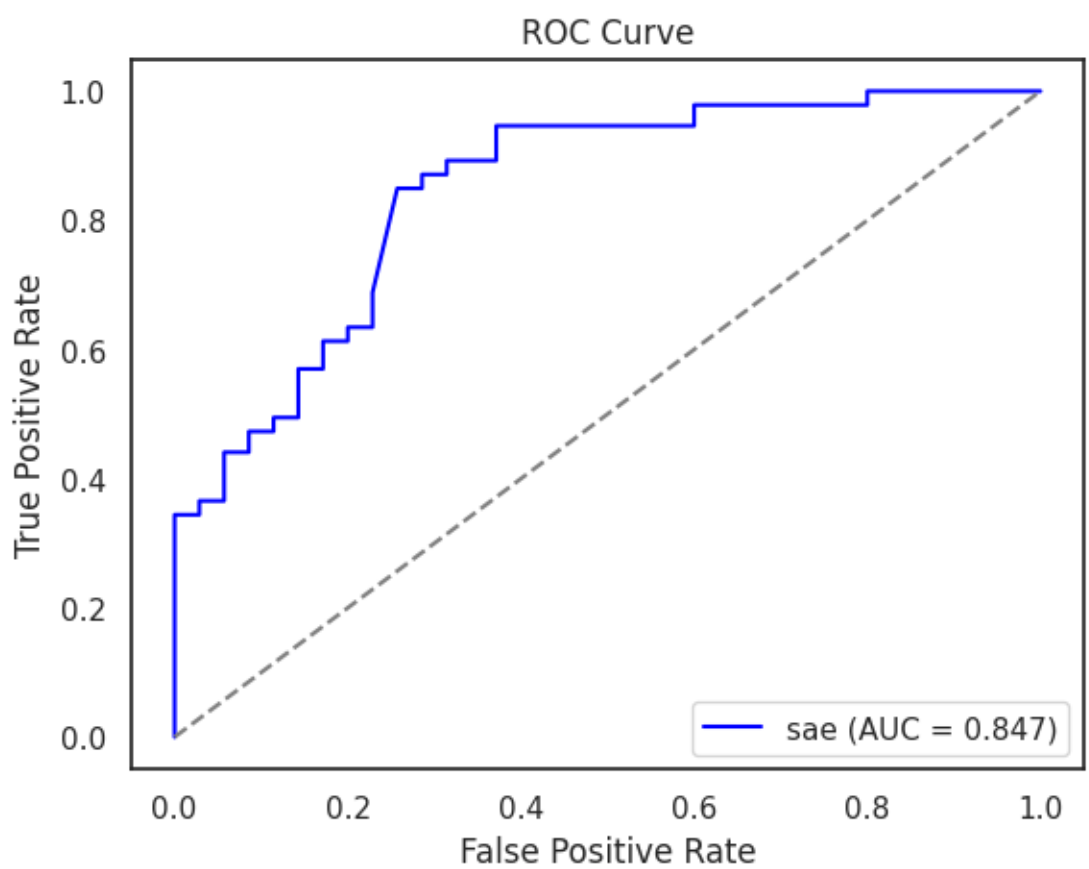
```
accuracy: 0.8397 - loss: 0.6230 - val_accuracy: 0.8431 - val_loss: 0.6121
Epoch 27/30
4/4              0s 25ms/step -
accuracy: 0.8238 - loss: 0.6180 - val_accuracy: 0.8431 - val_loss: 0.6015
Epoch 28/30
4/4              0s 25ms/step -
accuracy: 0.8278 - loss: 0.6050 - val_accuracy: 0.8627 - val_loss: 0.5897
Epoch 29/30
4/4              0s 23ms/step -
accuracy: 0.8603 - loss: 0.5826 - val_accuracy: 0.8627 - val_loss: 0.5767
Epoch 30/30
4/4              0s 28ms/step -
accuracy: 0.8878 - loss: 0.5682 - val_accuracy: 0.8627 - val_loss: 0.5599
```

[20]:
```
plt.figure()
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend()
plt.show()
```



[21]:
```python
loss, acc = sae.evaluate(xtest, ytestlabels, verbose=0)
print("loss:", loss)
print("accuracy:", acc)
# Predicciones (probabilidades)
yhat = sae.predict(xtest)

# Clases con umbral 0.5
yhat_class = np.where(yhat < 0.51, 0, 1).ravel()

# Tabla de contingencia (equivalente a table())
table = pd.crosstab(
    yhat_class,
    ytestlabels,
    rownames=["yhatclass"],
    colnames=["ytestlabels"]
)

print(table)
from sklearn.metrics import confusion_matrix, classification_report

# Asegúrate de que yhat_class y ytestlabels sean vectores 1D
yhat_class = yhat_class.ravel()  # si aún tiene shape (n,1)
ytestlabels = np.array(ytestlabels).ravel()

# Matriz de confusión
cm = confusion_matrix(ytestlabels, yhat_class)
print("Confusion matrix:\n", cm)

# Métricas completas (accuracy, precision, recall, f1)
print("\nClassification report:")
print(classification_report(ytestlabels, yhat_class))
# se ha predicho todo como positivo
```

```
loss: 0.5585865378379822
accuracy: 0.84375
4/4              0s 6ms/step
ytestlabels   0   1
yhatclass
0            22   9
1            13  84
Confusion matrix:
 [[22 13]
 [ 9 84]]

Classification report:
              precision    recall  f1-score   support

           0       0.71      0.63      0.67        35
```

```
           1       0.87      0.90      0.88        93

    accuracy                           0.83       128
   macro avg       0.79      0.77      0.78       128
weighted avg       0.82      0.83      0.82       128
```

[22]:
```python
from sklearn.metrics import roc_curve, auc
# yhat debe ser PROBABILIDAD, no clase
yhat_prob = yhat.ravel()   # equivalente a as.numeric(yhat)

# ROC
fpr, tpr, thresholds = roc_curve(ytestlabels, yhat_prob)
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr, color="blue", label=f"sae (AUC = {roc_auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", color="grey")  # diagonal random
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.show()
```